Project Networks: CPM and PERT

Planning an inspection-robot demonstration

The delivery question

A team is preparing an inspection-robot demonstration. Can it finish in 12 working days? Where would overtime actually save a project day?

Today's tools

Build a task network; calculate earliest and latest times; explain slack; compare crashing and uncertainty.

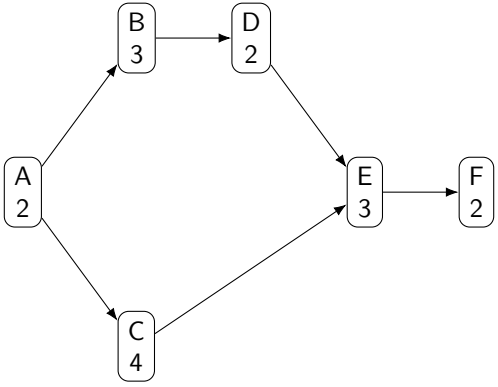
This is the hand-worked companion to Lab 5. Its six tasks are deliberately smaller than the laboratory's full robot project.

Start with a story, then a task table

ID	Deliverable	Predecessors	Days
A	Freeze design	none	2
B	Procure hardware	A	3
C	Build perception demo	A	4
D	Assemble robot	B	2
E	Integrate system	C,D	3
F	Validate demonstration	E	2

Time starts at day 0. A task can start when *all* predecessors finish. Durations use one consistent working-day calendar.

Draw dependencies, not just a list



The graph must have no directed cycle. An arrow means finish-to-start precedence with zero lag. It does not reserve an engineer or a workstation.

The critical path is the longest route

There are two complete start-to-finish paths:

$$A \to B \to D \to E \to F : 2 + 3 + 2 + 3 + 2 = 12,$$
$$A \to C \to E \to F : 2 + 4 + 3 + 2 = 11.$$

Both branches must finish before integration. Therefore the minimum project duration under precedence alone is **12 days**. The longest path controls completion. Finding a shortest path would answer the wrong question.

Forward pass: when can each task start?

Let ES_i and EF_i be earliest start and earliest finish.

$$ES_i = \max_{p \in \mathrm{pred}(i)} EF_p, \qquad EF_i = ES_i + d_i.$$

Set $ES = 0$ for tasks without predecessors. Process tasks in topological order, with every predecessor before its successor. For E: $ES_E = \max(EF_C, EF_D) = \max(6, 7) = 7$.

Check: Why is ES_E a maximum, not a sum?

Answer: C and D can overlap; E waits for the later finish.

Forward pass: all six rows

Task	Duration	*ES*	*EF*
A	2	0	2
B	3	2	5
C	4	2	6
D	2	5	7
E	3	7	10
F	2	10	12

The earliest project finish is the largest terminal *EF*, here 12. For multiple terminal tasks, a zero-duration Finish node makes the common endpoint explicit.

Backward pass: how late is still on time?

Use the earliest possible project finish $T = 12$ as the target for terminal tasks.

$$LF_i = \min_{s \in \mathrm{succ}(i)} LS_s, \qquad LS_i = LF_i - d_i.$$

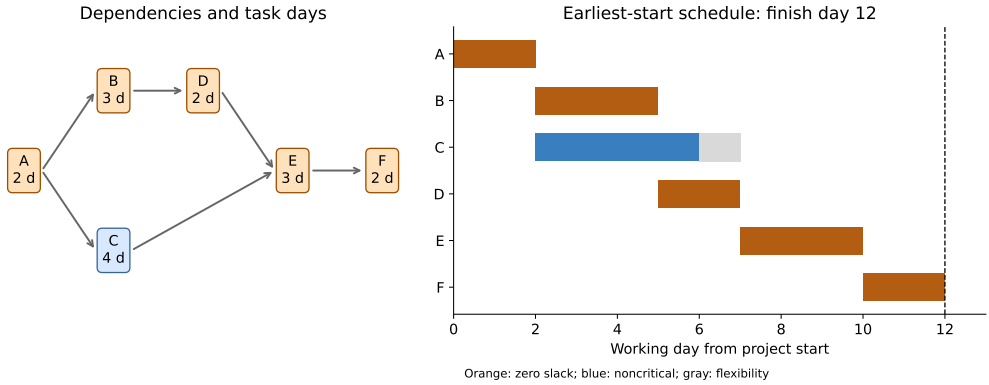
Work in reverse topological order; set terminal $LF = T$. For C: $LF_C = LS_E = 7$, so $LS_C = 7 - 4 = 3$.
For A: $LF_A = \min(LS_B, LS_C) = \min(2, 3) = 2$. The minimum protects *every* successor's latest start.

The complete CPM table

Task	ES	EF	LS	LF	Slack
A	0	2	0	2	0
B	2	5	2	5	0
C	2	6	3	7	1
D	5	7	5	7	0
E	7	10	7	10	0
F	10	12	10	12	0

Total slack is $LS - ES = LF - EF$. C may move from day 2 to day 3 without delaying day 12, if all other assumptions stay fixed. Slack is scheduling flexibility, not extra work duration.

Read the network and its schedule together



The same six tasks in both views. Orange tasks have zero slack; gray shows C's one day of flexibility. Overlap assumes sufficient resources. Dashed line: project finish at day 12.

A quick decision check

1. C becomes one day shorter. When does the project finish?
2. B becomes one day shorter. When does the project finish?
3. C is delayed two days at its start. When does the project finish?

Pause and calculate the two complete path lengths before looking at the answers.

Answers: changing tasks can change the bottleneck

1. C shorter: paths become 12 and 10; finish stays **12**.
2. B shorter: paths both become 11; finish is **11**, with *two* critical paths.
3. C starts at day 4: it finishes at day 8, E finishes at 11, and F at **13**.

The first day of C's delay consumes slack; the second delays the project. Slack values are conditional on the current network and durations, so recompute after a change.

Multiple critical paths require connected chains

After B is shortened to two days, both

$$A \to B \to D \to E \to F \quad \text{and} \quad A \to C \to E \to F$$

are critical. The project lasts 11 days and all tasks have zero slack. Listing all zero-slack tasks is not a path. Trace connected predecessor-successor chains satisfying $EF_i = ES_j$. Speeding up just C or just D now saves zero project days. Both branches must improve, or a shared task must improve.

Crashing buys shorter task durations

Assume each option reduces a task by exactly one day, at most once:

Task	Incremental cost (PLN)
B	100
C	40
D	150
E	200
F	300

Goal: finish by day 10 at minimum extra cost. These are hypothetical independent options. Real crashing may have diminishing returns, minimum durations, or incompatible staffing choices.

Work the crashing choice before optimizing

Buy B's one-day reduction for 100 PLN: finish falls from 12 to 11; both branches are now critical.

- Shorten shared E by one day: another 200 PLN, total 300 PLN, finish 10.
- Shorten C and D by one day each: another $40 + 150 = 190$ PLN, total **290 PLN**, finish 10.

Buying only the cheapest option C saves no time initially. Buying reductions greedily on one original critical path can miss the cheapest plan.

State the optimization model

Let $r_i \in \{0,1\}$ indicate buying an allowed reduction; $r_A = 0$. Let $s_i \geq 0$ be start times.

$$\begin{aligned} \min \quad & 100r_B + 40r_C + 150r_D + 200r_E + 300r_F \\ \text{s.t.} \quad & s_j \geq s_i + d_i - r_i && \forall (i,j) \text{ dependency}, \\ & s_F + d_F - r_F \leq 10. \end{aligned}$$

The notebook enumerates all $2^5 = 32$ allowed combinations, reruns CPM, and verifies that 290 PLN is optimal. Enumeration is practical here because the option set is tiny.

CPM does not create extra engineers

Suppose B and C require the same engineer and cannot overlap. B first: A $[0, 2]$, B $[2, 5]$, C $[5, 9]$, D $[5, 7]$, E $[9, 12]$, F $[12, 14]$. Finish: **14 days**. C first: A $[0, 2]$, C $[2, 6]$, B $[6, 9]$, D $[9, 11]$, E $[11, 14]$, F $[14, 16]$. Finish: **16 days**. The 12-day CPM result is a precedence-only lower bound. Resource-constrained scheduling must choose an order for competing tasks, just as the job-shop laboratory does.

Uncertain durations: what PERT adds

For each task, elicit optimistic a, most likely m, and pessimistic b, with $a \le m \le b$.

$$\mu = \frac{a + 4m + b}{6}, \qquad v \approx \left(\frac{b - a}{6}\right)^2.$$

These are traditional PERT moment estimates, not a theorem that any three guesses determine a unique distribution. Example for D: $(a, m, b) = (1, 2, 3)$ gives $\mu_D = 2$ days and approximate $v_D = 1/9$ days^2.

A hand-computable project risk estimate

For this example, use $(d_i - 1, d_i, d_i + 1)$ for every task. Means equal deterministic durations; the expected-duration critical path is A–B–D–E–F.

$$\mu_{\text{path}} = 12, \qquad v_{\text{path}} \approx 5/9, \qquad \sigma_{\text{path}} \approx 0.745.$$

With independent task times and this path treated as fixed, approximate its duration by a normal distribution:

$$P(T \leq 13) \approx \Phi\left(\frac{13 - 12}{0.745}\right) = \Phi(1.342) \approx 0.910.$$

Why 91 percent is only an approximation

- The project duration is the *maximum* of path durations; the critical path may switch.
- Shared tasks make the two path durations dependent even when individual task durations are independent.
- Task times may be correlated through suppliers, weather, or shared people. Variances then include covariance terms.
- A normal approximation can be poor with a few highly skewed tasks or bounded durations.

In general $E[\max(P_1, P_2)] \geq \max(E[P_1], E[P_2])$. CPM on mean task times need not give the mean project finish.

Monte Carlo: sample tasks, recompute the network

For each simulated project:

1. Draw one duration for every task from the specified distributions.
2. Reuse that draw wherever the task appears in a path.
3. Run the forward pass and record the project finish.

Estimate deadline probability as the fraction of finishes on time. Report the assumed distributions, sample size, seed, and Monte Carlo standard error $\sqrt{\hat{p}(1-\hat{p})/N}$. Simulation explores the assumed model; it does not validate optimistic input estimates.

A subtle distribution issue in the lab

For $b > a$, the lab simulation uses a scaled beta family with

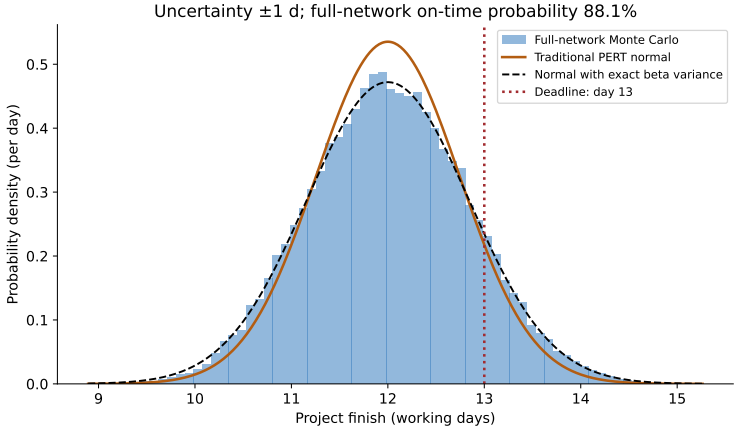
$$\alpha = 1 + 4\frac{m-a}{b-a}, \qquad \beta = 1 + 4\frac{b-m}{b-a}.$$

Its mean equals $(a + 4m + b)/6$, but its variance is

$$(b-a)^2 \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}.$$

If $a = m = b$, use that fixed duration and zero variance instead.
For $(1, 2, 3)$, $\alpha = \beta = 3$: variance is $1/7$, whereas traditional PERT uses $1/9$. A difference from normal PERT can reflect *both* distribution assumptions and path switching. The companion reports both.

Compare three explicit uncertainty models



Same six-task example, independent beta task times, 30,000 trials, seed 505. Both normal curves use one fixed expected critical path; the histogram recomputes the network finish for every draw.

Interactive companion: make a prediction first

Open `lecture_5_cpm_pert.ipynb`.

1. Shorten B by one day; inspect both critical paths.
2. Enumerate crashing plans and explain the 290 PLN optimum.
3. Increase duration uncertainty; compare fixed-path normal estimates with Monte Carlo that recomputes the finish.
4. Compare the textbook variance and the actual beta variance separately.

All plots and tables have direct function calls for environments without widget support.

Risk mitigation is a different decision from crashing

An interface review may add a day yet reduce integration rework. Evaluate the revised dependencies *and* revised duration distribution together. Compare deterministic finish, simulated deadline probability, and incremental cost. A task need not shorten the baseline schedule to be worthwhile. Check: Does a simulated 90% on-time probability mean a guaranteed deadline?

Answer: no. It is conditional on task estimates, dependencies, resource assumptions, and the chosen distribution model.

Transfer to Lab 5

The upstream lab scales this approach to an autonomous inspection-robot demonstration, with component suppliers, crashing choices, interface review, and Monte Carlo.

- Check IDs, units, predecessor references, and acyclicity first.
- Recompute CPM after each supplier or crashing change.
- Distinguish task days saved from project days saved.
- Report a connected critical path; note when several exist.
- Present risk estimates with the distribution and resource assumptions.

Sources and reproducibility

- NASA: Schedule Management Overview and PP&C glossary: integrated logic networks and the critical path method.
- SciPy: beta distribution: distribution parameterization and moments.
- Lab 5: project networks, CPM, PERT, and Gantt charts. Reviewed upstream snapshot: `3c1b7a7`.

Companion data and costs are synthetic. Its complete six-task network is specified in this deck and notebook, with no external data download.